Plan Testów dla aplikacji Practice Software Testing

	Imię i nazwisko	Kontakt (adres e-mail)
Współautor	Dominik Gałkowski	247659
Współautor	Jan Śladowski	247806

Spis treści

1. Wstęp 2
2. Terminologia i definicje 2
3. Zakres dokumentu 3
4. Opis testowanego systemu 3
5. Zakres planowanych testów 4
6. Założenia i ograniczenia 4
7. Interesariusze i komunikacja 5
8. Strategia testowania 5
9. Artefakty z procesu testowego 7
10. Opis środowiska testowego 7
11. Harmonogram testowania 7
12. Implementacja testów 8
13. Lista scenariuszy testowych 9

Historia zmian				
Rewizja	Data	Modyfikujący	Zmodyfikowane paragrafy	Komentarz
	14.11.2025r.	Jan Śladowski	Lista scenariuszy testowych	Aktualizacja scenariuszy testowych

1. Wstęp

Niniejszy dokument przedstawia plan testów dla aplikacji Practice Software Testing, przykładowego systemu e-commerce opracowanego w celach edukacyjnych do nauki testowania manualnego i automatycznego. Plan opisuje zakres, strategię, środowisko, harmonogram oraz metody testowania. W kolejnych rozdziałach dokumentu opisano:

- teorię mającą za zadanie określić kontekst testowania,
- koncepcję mającą określić optymalne podejście do testowania systemu oraz odpowiednie techniki testowania,
- założenia mające określić plan, harmonogram i technologie testowania,

2. Terminologia i definicje

Termin / skrót	Opis
System	Aplikacja webowa Practice Software Testing, dostępna przez przeglądarkę internetową.
Frontend	Część aplikacji zbudowana w Angular – interfejs użytkownika.
Backend	Warstwa serwerowa oparta o REST API, odpowiadająca za logikę biznesową.
API	Punkty dostępowe przez które frontend komunikuje się z backendem.
Defekt	Niezgodność działania systemu z jego wymaganiami.
Plan testów	Niniejszy dokument opisujący zakres, metody, zasoby oraz harmonogram zamierzonych czynności testowych.
BDD	Behavior-Driven Development - metodyka tworzenia oprogramowania i testów, w której zachowanie systemu jest opisywane w sposób czytelny dla osób technicznych i nietechnicznych, z wykorzystaniem prostego języka naturalnego.

3. Zakres dokumentu

Niniejszy plan testów obejmuje cały system, w tym zarówno warstwę frontendową, jak i backendową (API). Dokument obejmuje wszystkie fazy testowania systemu, ich organizację, strategię, środowisko i harmonogram. ale nie zawiera szczegółów dotyczących przypadków testowych.

4. Opis testowanego systemu

Aplikacja Practice Software Testing to platforma e-commerce stworzona w celach edukacyjnych, umożliwiająca naukę testowania oprogramowania w środowisku rzeczywistego systemu webowego. System symuluje pełen proces zakupowy użytkownika - od rejestracji i logowania, przez dodawanie produktów do koszyka, aż po wystawianie faktur i zarządzanie kontem.

Aplikacja działa w architekturze klient–serwer i składa się z następujących głównych komponentów:

1. Frontend (Angular)
    a. Odpowiada za interfejs użytkownika i interakcję z serwerem.
    b. Komunikuje się z backendem poprzez zapytania HTTP REST API.
2. Backend (Laravel)
    a. Realizuje logikę biznesową systemu i udostępnia REST API dla frontendu.
    b. Obsługuje żądania od frontendu.
3. Baza danych (MariaDB)
    a. Przechowuje dane o produktach, użytkownikach, fakturach i zamówieniach.

Komponenty systemu:

- Brand – zarządzanie markami produktów (dodawanie, edycja, usuwanie)
- Card – obsługa funkcji koszyka zakupowego użytkownika,
- Category – zarządzenie kategoriami produktu,
- Contact – obsługa formularzy kontaktowych i wiadomości,
- Favorite – zarządzanie listą ulubionych produktów,
- Invoice – tworzenie i przetwarzanie faktur,
- Product – przetwarzanie logiki związanej z produktami,
- Report – generowanie raportów sprzedaży,
- TOTP – obsługa Time-based One-Time Password,
- User – rejestracja, logowanie, autoryzacja i zarządzanie kontami użytkowników.

5. Zakres planowanych testów

Testowaniu zostaną poddane komponenty backendowy, udostępniający interfejs REST API, oraz komponent frontendowy.

W ramach procesu testowego zostaną przeprowadzone testy jednostkowe, obejmujące logikę biznesową backendu oraz wybrane funkcje frontendowe. Celem tych testów jest weryfikacja poprawności działania pojedynczych metod, kontrolerów i serwisów API.

Zostaną również wykonane testy integracyjne, które sprawdzą współdziałanie modułów backendu (np. komunikację pomiędzy kontrolerami a bazą danych). Ich celem jest potwierdzenie spójności przepływu danych oraz poprawności komunikacji między komponentami.

Kolejnym etapem będą testy akceptacyjne, opracowane w oparciu o scenariusze zapisane w formacie BDD. Mają one na celu weryfikację, czy aplikacja spełnia wymagania użytkownika i jest gotowa do wdrożenia.

W procesie testowania zostaną wykorzystane testy funkcjonalne, które potwierdzą realizację wymagań funkcjonalnych, oraz testy strukturalne, służące ocenie poprawności wewnętrznej implementacji.

6. Założenia i ograniczenia

W procesie testowania systemu przyjęto następujące założenia oraz zidentyfikowano ograniczenia technologiczne i organizacyjne:

Założenia technologiczne:

- testy będą wykonywane w środowisku testowym z wykorzystaniem kontenerów w Dockerze,
- dane testowe zostaną utworzone automatycznie,
- backend API jest stabilny i udostępnia interfejs REST API,
- testy UI będą uruchamiane w przeglądarkach Chrome,
- testy wydajnościowe są wyłączone z zakresu.

Założenia procesowe:

- testy automatyczne będą implementowane i wykonywane w ramach Sprintu 5, zgodnie z harmonogramem projektu.

Ograniczenia:

- testy wydajnościowe, obciążeniowe i bezpieczeństwa nie są objęte zakresem bieżącego planu testów,
- środowisko testowe nie jest tożsame z produkcyjnym i może różnić się konfiguracją bazy danych oraz dostępnością usług zewnętrznych.

7. Interesariusze i komunikacja

Interesariuszami procesu testowanego dla systemu są:

- Testerzy systemu
- Prowadzący zajęcia Jakości w procesie wytwarzania oprogramowania

Komunikacja w ramach procesu testowania odbywać będzie się następującymi kanałami komunikacji:

- co dwa tygodnie spotkania między testerami a prowadzącym zajęcia,
- bezpośrednio za pomocą komunikatora MS Teams,
- bezpośrednio za pomocą komunikatora Discord.

8. Strategia testowania

Proces testowy dla systemu zakłada pełną automatyzację przypadków testowych w ramach trzech głównych etapów testowych. Testowanie zostanie przeprowadzone iteracyjnie zgodnie z trzema etapami.

Etap I – testy API

- Implementacja testów automatycznych dla endpointów REST,
- Testy obejmą poprawność odpowiedzi i statusów HTTP, walidację struktury odpowiedzi JSON, weryfikację integralności danych,
- Testy będą realizowane poprzez trzy warianty:
    - happy path - testy poprawnego przepływu,
    - negative testing - testy błędnych danych i niepoprawnych żądań,
    - destructive testing - testy odporności systemu na nietypowe dane wejściowe.
- Poziomy testów: jednostkowe i integracyjne,
- Typy testów: funkcjonalne i strukturalne.

Etap II – testy UI

- Automatyzacja testów interfejsu użytkownika w przeglądarce,
- Scenariusze testowe obejmują m.in. logowania i rejestrację użytkownika lub operacje na koszyku zakupowym,

- testy zostaną zorganizowane w strukturze Page Object Model (POM),
- Poziomy testów: jednostkowe i integracyjne,
- Typy testów: funkcjonalne i strukturalne.

Etap III – refaktoryzacja w podejściu BDD.

- transformację istniejących testów automatycznych w podejście BDD.
- każdy scenariusz będzie zawierał strukturę:
    - Given – stan początkowy,
    - When – akcja użytkownika,
    - Then – oczekiwany rezultat.

Wykryte defekty podlegają następującej klasyfikacji.

Klasyfikacja	**Wpływ na system**	**Priorytet naprawy**
Najwyższy	Defekt blokujący kluczową funkcję, co uniemożliwia wykorzystanie systemu.	Najwyższy priorytet naprawy spowodowany potencjalnymi opóźnieniami w procesie projektowym.
Wysoki	Kluczowa funkcja lub wymaganie zostało błędnie zaimplementowane.	Potencjał na opóźnienie procesu projektowego, ale naprawa defektu może zostać odłożona za sprawą możliwych obejść.
Średni	Drugorzędna funkcja bądź wymaganie zostało źle zaimplementowane. System może być nadal używany.	Naprawa defektu powinna zostać przeprowadzona w ramach najbliższego Sprintu.
Niski	Niewielkie niezgodności zostały wykryte. System może być użyty bez restrykcji.	Naprawa powinna zostać przeprowadzona, jeśli będzie dostępny na nią czas.
Najniższy	Nieistotny defekt został znaleziony. Nie wpływa na działanie systemu.	Kiedy czas pozwoli.

Warunki akceptacji:

- wszystkie testy automatyczne zakończone statusem PASS dla scenariuszy krytycznych (Happy Path).
- brak otwartych defektów o klasyfikacji Najwyższy lub Wysoki.

- pokrycie testami automatycznymi:
    - co najmniej 80% endpointów API,
    - co najmniej 70% kluczowych ścieżek użytkownika w UI.
- zgodność wyników testów z dokumentacją wymagań i specyfikacją Swagger.

9. Artefakty z procesu testowego

W wyniku realizacji procesu testowego dla systemu powstaną następujące artefakty dokumentujące przebieg, wyniki i automatyzację testów:

- plan testów (niniejszy dokument),
- dokumentacja przypadków testowych - zbiór opisów przypadków testowych obejmujących zarówno testy API, jak i UI, wraz z ich celami, danymi wejściowymi, krokami wykonania oraz oczekiwanymi rezultatami,
- dokumentacja scenariuszy BDD - scenariusze zapisane w formacie Gherkin w plikach .feature, opisujące zachowanie systemu w sposób czytelny dla użytkowników biznesowych.
- repozytoria testów automatycznych - kod testów automatycznych będzie przechowywany w zdalnych repozytoriach:
    - /api – testy REST API,
    - /ui – testy interfejsu,
    - /bdd – pliki .feature Gherkin,
- raport z procesu testowego.

10. Opis środowiska testowego

Środowisko testowe dla systemu zostało w pełni zbudowane i uruchomione w oparciu o kontenery Docker, co zapewnia spójność konfiguracji, izolację komponentów oraz łatwą replikację środowiska na różnych stanowiskach testerów.

Aplikacja, obejmująca warstwę frontendową, backendową oraz bazę danych, jest uruchamiana w trybie deweloperskim przy użyciu pliku konfiguracyjnego docker-compose.yml. Konfiguracja ta umożliwia równoczesne uruchomienie wszystkich usług niezbędnych do testów w ramach jednej sieci kontenerów.

11. Harmonogram testowania

Etap	Zakres	Czas trwania	Odpowiedzialny

Etap I	Testy automatyczne API + dokumentacja przypadków.	2 tygodnie	Dominik Gałkowski, Jan Śladowski
Etap II	Testy automatyczne UI + dokumentacja przypadków.	2 tygodnie	Dominik Gałkowski, Jan Śladowski
Etap III	Refaktoryzacja testów na podejście BDD.	2 tygodnie	Dominik Gałkowski, Jan Śladowski

12. Implementacja testów

Do przeprowadzenia testów zostaną wykorzystane następujące technologie:

- Postman,
- Newman,
- Selenium Webdriver + Java 21,
- Gherkin i Cucumber,
- GitHub - zdalne repozytorium przechowujące kod testów.

Repozytorium testowe i nazewnictwo będzie trzymało się następujących konwencji:

- Testy automatyczne API przechowywane w ramach Postman Collection, z nazewnictwem „/api”,
- Testy automatyczne UI przechowywane w ramach zdalnego repozytorium Git, z konwencją nazewniczą „/ui”,
- Testy w metodologii BDD przechowywane w ramach zdalnego repozytorium Git, z konwencją nazewniczą „/bdd”.

13. Lista scenariuszy testowych

Scenariusze testowe zostały przygotowane na podstawie dokumentacji Swagger dla systemu i podzielone na sekcje odpowiadające danym komponentom.

1. Brand
    a. Testy dla dodawania nowej marki do bazy danych.
    b. Testy dla otrzymania marki na podstawie podanego zapytania.
2. Cart
    a. Testy dla stworzenia nowej karty.
    b. Testy dla dodania produktu do karty.
    c. Testy dla usunięcia produktu z karty.
3. Category
    a. Testy dla dodania nowej kategorii do bazy danych.
    b. Testy dla częściowej aktualizacji konkretnej kategorii.
4. Contact
    a. Testy dla wysłania nowej wiadomości.
5. Favorite
    a. Testy dla dodania nowego ulubionego produktu.
    b. Testy dla usunięcia ulubionego produktu.
6. Invoice
    a. Testy dla dodania nowej faktury.
    b. Testy dla edycji konkretnej faktury.
7. Product
    a. Testy dla dodania nowego produktu.
    b. Testy dla otrzymania konkretnego produktu.
    c. Testy dla edycji konkretnego produktu.
    d. Testy dla otrzymania produktu na podstawie podanego zapytania.
8. User
    a. Testy dla rejestracji nowego użytkownika.
    b. Testy dla logowania użytkownika.
    c. Testy dla otrzymania informacji o aktualnym użytkowniku.